

Quantium Retail Analytics

Task 2 — Experimentation and Uplift Testing

Matched-control evaluation of trial stores 77, 86, and 88

Executive recommendation

Roll out only where the evidence is repeatable and commercially meaningful. Store-level results differ, so a blanket national rollout is not supported by this three-store trial.

Trial Store	Control Store	Match Score	Sales Uplift	Customer Uplift	Txn Per Customer Uplift	Recommendation
77	233	0.968	+26.2%	+23.1%	-2.0%	ROLL OUT
86	155	0.943	+13.2%	+13.5%	+0.7%	DO NOT ROLL OUT / INVESTIGATE
88	237	0.803	+12.1%	+6.4%	+6.4%	ROLL OUT

Store 77: The sales uplift exceeds normal pre-trial variation; proceed to a controlled expansion while monitoring margin and repeat behaviour.

Store 86: The uplift was not sufficiently consistent across sales and customers; do not roll it out in its current form, and retest if operational evidence warrants it.

Store 88: The sales uplift exceeds normal pre-trial variation; proceed to a controlled expansion while monitoring margin and repeat behaviour.

Decision rule

A trial is successful when positive sales uplift clears both the one-sided 95% threshold and a 10% commercial-materiality floor in at least two of the three trial months. The materiality floor prevents small, method-sensitive differences from driving rollout decisions. Customer count and transactions per customer diagnose the change.

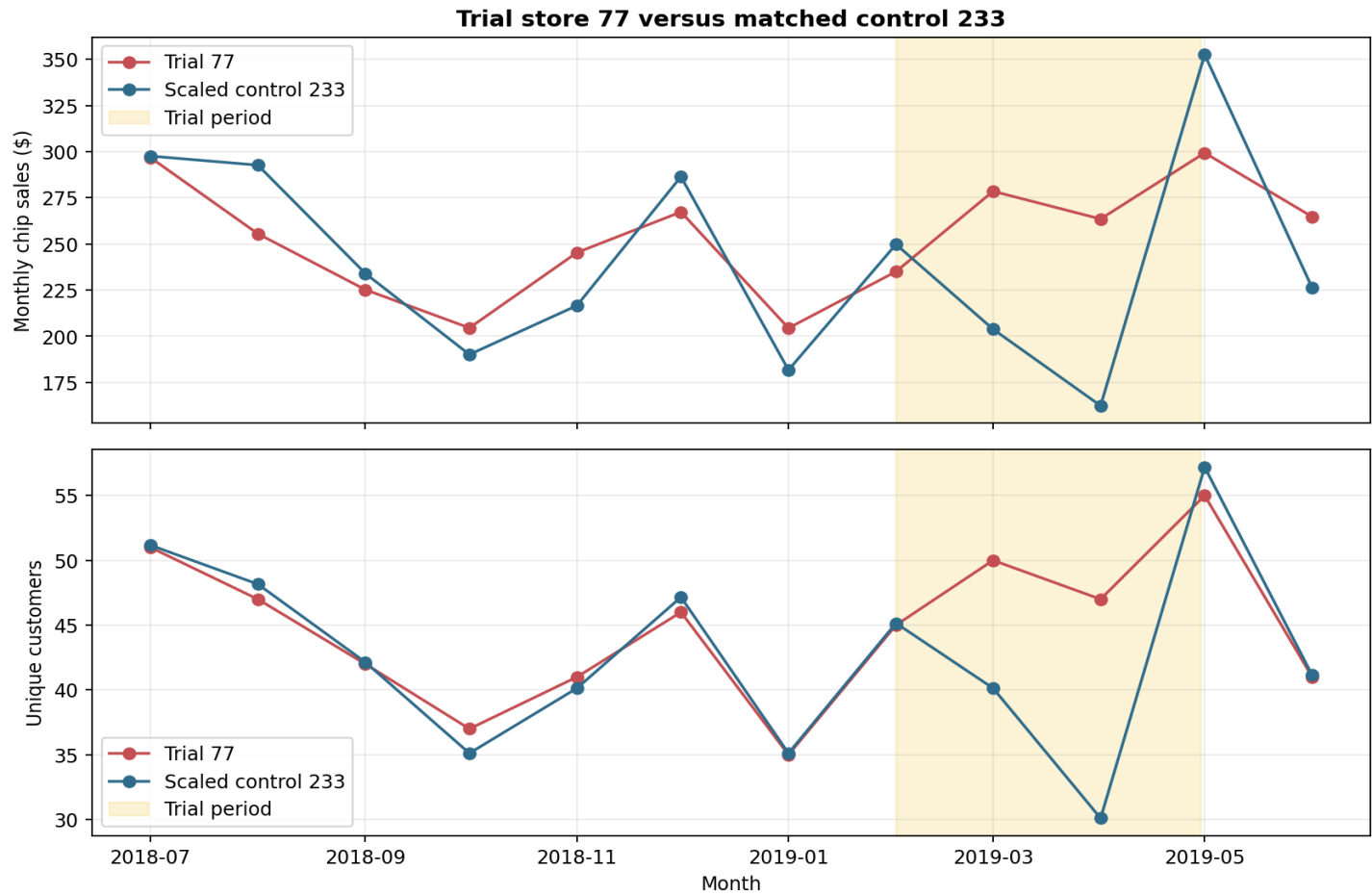
Control-store selection

Candidate controls required complete observations for all seven pre-trial months (July 2018–January 2019). Each candidate was scored equally on sales correlation, customer correlation, sales magnitude similarity, and customer magnitude similarity. Trial stores were excluded from the control pool.

Trial Store	Control Store	Sales Corr	Customer Corr	Score
77	233	0.904	0.990	0.968
77	41	0.783	0.844	0.892
77	17	0.843	0.747	0.859
77	254	0.577	0.916	0.839
77	115	0.689	0.719	0.827
86	155	0.878	0.943	0.943
86	109	0.788	0.771	0.872
86	114	0.734	0.855	0.862
86	138	0.760	0.750	0.841
86	225	0.618	0.734	0.819
88	237	0.308	0.947	0.803
88	178	0.732	0.939	0.799
88	69	0.450	0.816	0.713
88	113	0.496	0.863	0.708
88	201	0.493	0.511	0.702

Using both trend and scale prevents selecting a store that moves similarly but operates at a materially different level. The top-ranked store for each trial is used below.

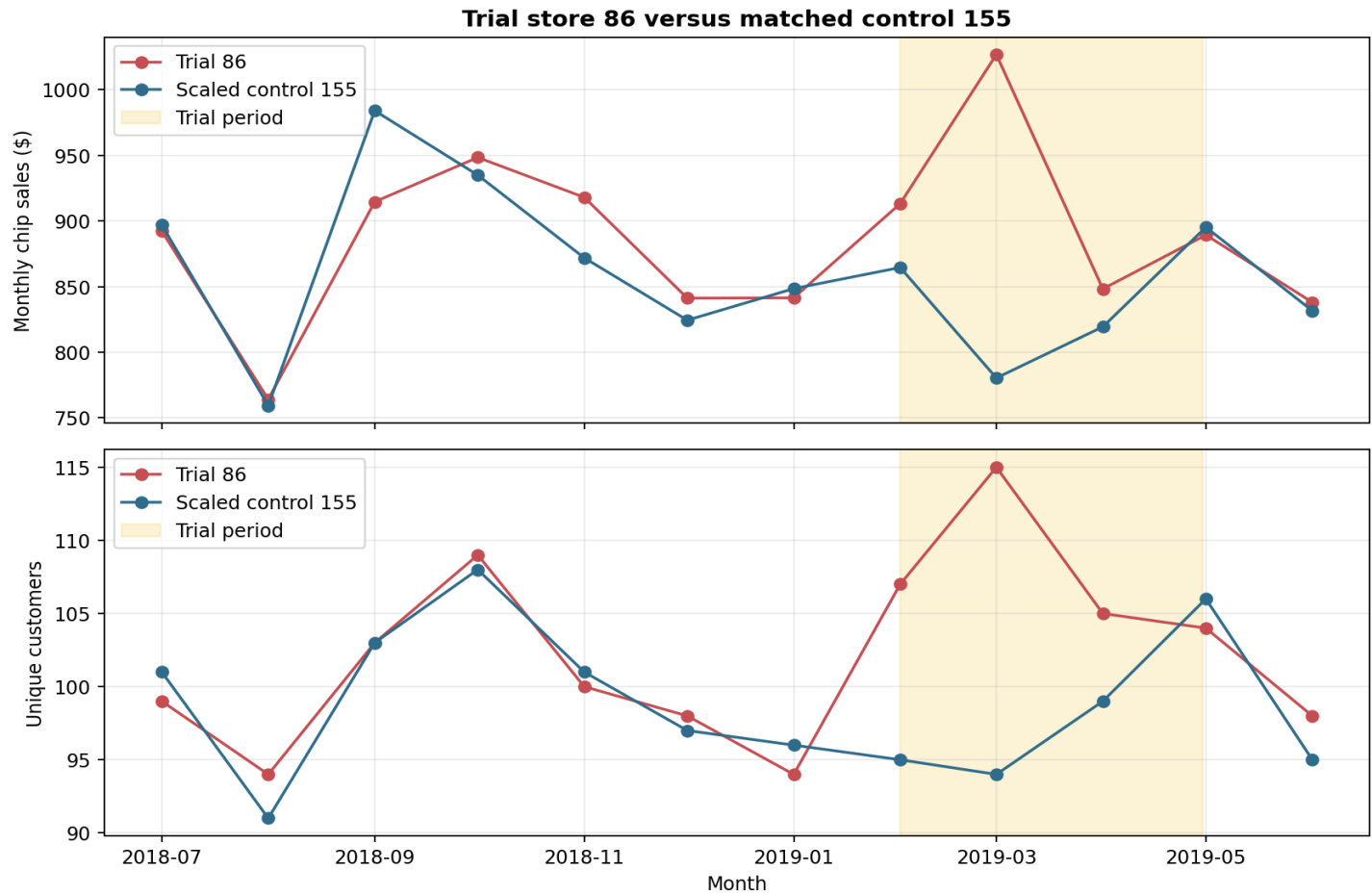
Trial store 77 vs control 233



Month	Trial Sales	Control Sales	Sales Uplift	Trial Customers	Control Customers	Customer Uplift
2019-02	\$235	\$250	-5.9%	45	45	-0.3%
2019-03	\$278	\$204	+36.7%	50	40	+24.6%
2019-04	\$264	\$162	+62.3%	47	30	+56.1%

Trial-period sales uplift: +26.2%; customer uplift: +23.1%. Recommendation: **ROLL OUT**.

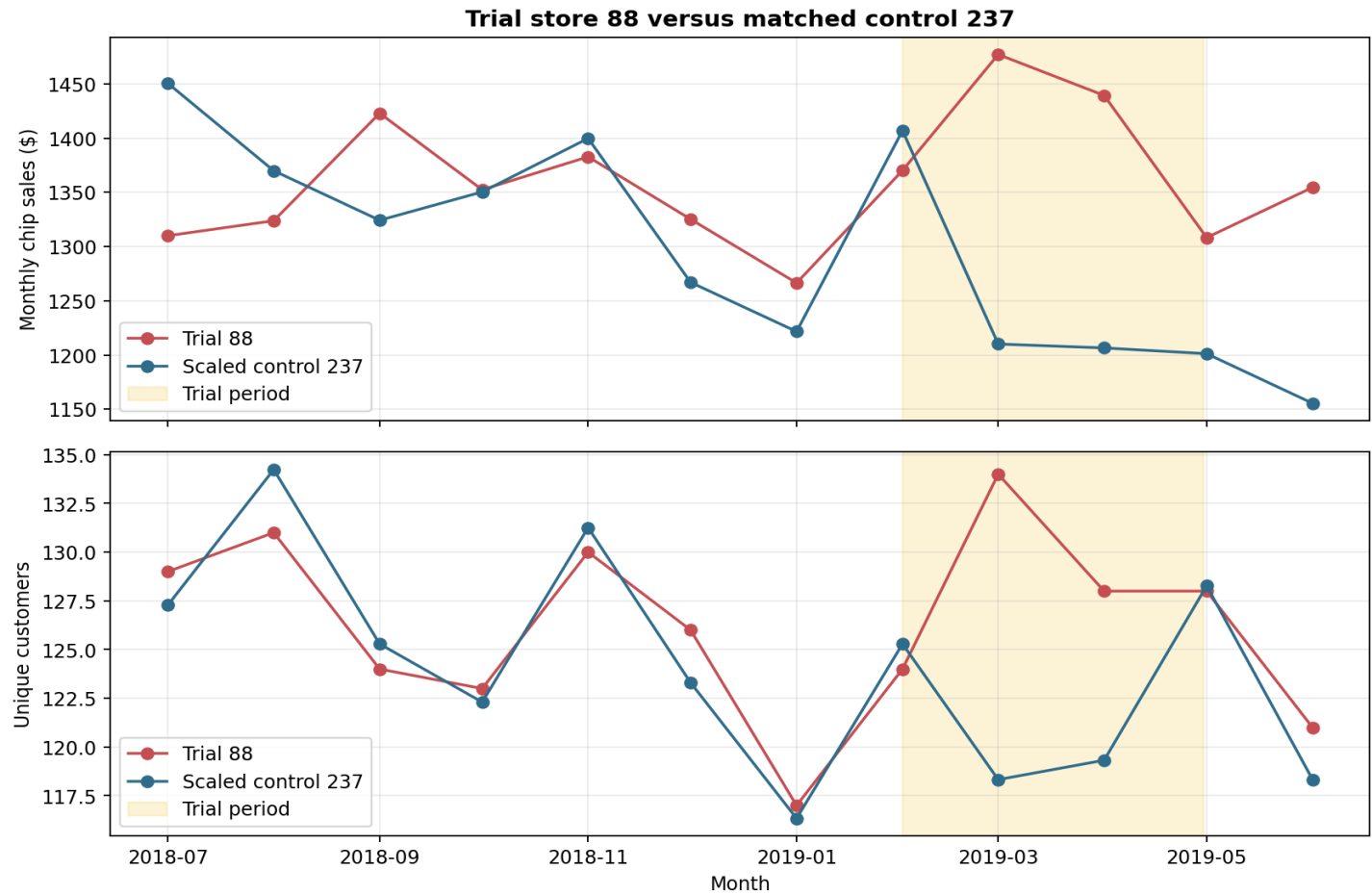
Trial store 86 vs control 155



Month	Trial Sales	Control Sales	Sales Uplift	Trial Customers	Control Customers	Customer Uplift
2019-02	\$913	\$865	+5.6%	107	95	+12.6%
2019-03	\$1,027	\$780	+31.6%	115	94	+22.3%
2019-04	\$848	\$819	+3.5%	105	99	+6.1%

Trial-period sales uplift: +13.2%; customer uplift: +13.5%. Recommendation: **DO NOT ROLL OUT / INVESTIGATE.**

Trial store 88 vs control 237



Month	Trial Sales	Control Sales	Sales Uplift	Trial Customers	Control Customers	Customer Uplift
2019-02	\$1,370	\$1,407	-2.6%	124	125	-1.0%
2019-03	\$1,477	\$1,210	+22.1%	134	118	+13.2%
2019-04	\$1,439	\$1,206	+19.3%	128	119	+7.3%

Trial-period sales uplift: **+12.1%**; customer uplift: **+6.4%**. Recommendation: **ROLL OUT**.

Limitations and next steps

This observational matched-control design reduces—but cannot eliminate—confounding. The dataset lacks margin, promotion, stock availability, store format, local events, and competitor activity. Only three trial months and three trial stores are available, so statistical power and generalisability are limited.

Before wider rollout, verify gross-margin uplift, check that results persist after promotions end, review operational execution, and expand with randomised or staggered test/control assignment. Track sales, customers, transactions per customer, units per transaction, and price per unit.

Appendix: complete Python code

```
"""Quantium Task 2: experimentation and uplift testing.
```

```

Selects matched control stores for trial stores 77, 86, and 88 using pre-trial
monthly sales and customer similarity, evaluates February-April 2019 uplift, and
creates a client-ready PDF containing results, charts, and complete source code.
"""
```

```

from pathlib import Path
import textwrap

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4, landscape
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Image, Table, TableStyle, Preformatted
from scipy.stats import t as student_t
```

```

ROOT = Path(__file__).resolve().parent
INPUT = Path.home() / "Downloads" / "QVI_data.csv"
OUTPUT_PDF = ROOT / "Quantium_Task2_Experimentation_and_Uplift_Testing.pdf"
OUTPUT_RESULTS = ROOT / "QVI_trial_results.csv"
CHART_DIR = ROOT / "task2_charts"
CHART_DIR.mkdir(exist_ok=True)
TRIAL_STORES = [77, 86, 88]
PRE_START, PRE_END = pd.Timestamp("2018-07-01"), pd.Timestamp("2019-01-31")
TRIAL_START, TRIAL_END = pd.Timestamp("2019-02-01"), pd.Timestamp("2019-04-30")
```

```

def page_number(canvas, doc):
    canvas.saveState()
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(colors.grey)
    canvas.drawRightString(285 * mm, 8 * mm, f"Page {doc.page}")
    canvas.restoreState()
```

```

def table(df, widths=None, font=7.5):
    shown = df.copy()
    shown.columns = [str(c).replace("_", " ").title() for c in shown.columns]
    obj = Table([shown.columns.tolist()] + shown.astype(str).values.tolist(), colwidths=widths, repeatRows=1)
    obj.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#17365D")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
        ("FONTSIZE", (0, 0), (-1, -1), font),
        ("GRID", (0, 0), (-1, -1), .25, colors.HexColor("#B8C2CC")),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F2F5F7")]),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
        ("LEFTPADDING", (0, 0), (-1, -1), 4), ("RIGHTPADDING", (0, 0), (-1, -1), 4),
    ]))
    return obj
```

```

def corr_score(pivot, trial, candidate):
    return float(pivot[trial].corr(pivot[candidate]))
```

```

def magnitude_score(pivot, trial, candidate):
    """Scale average relative distance to [0,1] across candidate stores later."""
    denom = (pivot[trial].abs() + pivot[candidate].abs()) / 2
    return float(((pivot[trial] - pivot[candidate]).abs() / denom.replace(0, np.nan)).mean())
```

```

def select_controls(pre, eligible):
    sales = pre.pivot(index="YEARMONTH", columns="STORE_NBR", values="TOTAL_SALES")
    customers = pre.pivot(index="YEARMONTH", columns="STORE_NBR", values="CUSTOMERS")
    rows = []
    for trial in TRIAL_STORES:
        candidates = [s for s in eligible if s != trial and s not in TRIAL_STORES]
        # The sample solution standardises absolute distance across candidate
        # stores within each month, then averages the monthly similarities.
        def monthly_magnitude(pivot):
```



```

distances = pivot[candidates].sub(pivot[trial], axis=0).abs()
row_min = distances.min(axis=1)
row_max = distances.max(axis=1)
similarities = 1 - distances.sub(row_min, axis=0).div((row_max - row_min).replace(0, np.nan), axis=0)
return similarities.mean(axis=0)

sales_magnitude = monthly_magnitude(sales)
customer_magnitude = monthly_magnitude(customers)
raw = []
for candidate in candidates:
    raw.append({
        "TRIAL_STORE": trial, "CONTROL_STORE": candidate,
        "SALES_CORR": corr_score(sales, trial, candidate),
        "CUSTOMER_CORR": corr_score(customers, trial, candidate),
        "SALES_MAGNITUDE": float(sales_magnitude[candidate]),
        "CUSTOMER_MAGNITUDE": float(customer_magnitude[candidate]),
    })
scores = pd.DataFrame(raw)
scores["SCORE"] = (
    scores["SALES_CORR"] + scores["CUSTOMER_CORR"] +
    scores["SALES_MAGNITUDE"] + scores["CUSTOMER_MAGNITUDE"]
) / 4
rows.append(scores.sort_values("SCORE", ascending=False))
return pd.concat(rows, ignore_index=True)

def plot_comparison(monthly, trial, control, scale_sales, scale_customers):
    subset = monthly[monthly["STORE_NBR"].isin([trial, control]).copy()]
    wide_sales = subset.pivot(index="MONTH", columns="STORE_NBR", values="TOTAL_SALES")
    wide_customers = subset.pivot(index="MONTH", columns="STORE_NBR", values="CUSTOMERS")
    fig, axes = plt.subplots(2, 1, figsize=(10, 7), sharex=True)
    axes[0].plot(wide_sales.index, wide_sales[trial], marker="o", label=f"Trial {trial}", color="#C44E52")
    axes[0].plot(wide_sales.index, wide_sales[control] * scale_sales, marker="o", label=f"Scaled control {control}",
        color="#2F6B8A")
    axes[1].plot(wide_customers.index, wide_customers[trial], marker="o", label=f"Trial {trial}", color="#C44E52")
    axes[1].plot(wide_customers.index, wide_customers[control] * scale_customers, marker="o", label=f"Scaled control
{control}", color="#2F6B8A")
    for ax, ylabel in zip(axes, ["Monthly chip sales ($)", "Unique customers"]):
        ax.axvspan(TRIAL_START, TRIAL_END, color="#F4D35E", alpha=.25, label="Trial period")
        ax.set_ylabel(ylabel)
        ax.grid(alpha=.25)
        ax.legend(loc="best")
    axes[0].set_title(f"Trial store {trial} versus matched control {control}", weight="bold")
    axes[1].set_xlabel("Month")
    fig.tight_layout()
    path = CHART_DIR / f"trial_{trial}_comparison.png"
    fig.savefig(path, dpi=170, bbox_inches="tight")
    plt.close(fig)
    return path

def main():
    data = pd.read_csv(INPUT, parse_dates=["DATE"])
    data["MONTH"] = data["DATE"].dt.to_period("M").dt.to_timestamp()
    data["YEARMONTH"] = data["DATE"].dt.strftime("%Y%m").astype(int)
    monthly = data.groupby(["STORE_NBR", "MONTH", "YEARMONTH"]).agg(
        TOTAL_SALES=("TOT_SALES", "sum"),
        CUSTOMERS=("LYLT_CARD_NBR", "nunique"),
        TRANSACTIONS=("TXN_ID", "nunique"),
        UNITS=("PROD_QTY", "sum"),
    ).reset_index()
    monthly["TXNS_PER_CUSTOMER"] = monthly["TRANSACTIONS"] / monthly["CUSTOMERS"]
    monthly["UNITS_PER_TXN"] = monthly["UNITS"] / monthly["TRANSACTIONS"]
    monthly["AVG_PRICE_PER_UNIT"] = monthly["TOTAL_SALES"] / monthly["UNITS"]

    pre = monthly[(monthly["MONTH"] >= PRE_START) & (monthly["MONTH"] <= PRE_END)].copy()
    # Controls must trade in every month of the full 12-month observation window.
    required_months = monthly["YEARMONTH"].nunique()
    eligible = monthly.groupby("STORE_NBR")["YEARMONTH"].nunique()
    eligible = eligible[eligible.eq(required_months)].index.tolist()
    scores = select_controls(pre, eligible)
    best = scores.groupby("TRIAL_STORE", as_index=False).first()

    result_rows, charts = [], []
    monthly_details = []
    for row in best.iteruples():
        trial, control = int(row.TRIAL_STORE), int(row.CONTROL_STORE)
        t_pre = monthly[(monthly.STORE_NBR == trial) & monthly.MONTH.between(PRE_START, PRE_END)]
        c_pre = monthly[(monthly.STORE_NBR == control) & monthly.MONTH.between(PRE_START, PRE_END)]
        scale_sales = t_pre.TOTAL_SALES.sum() / c_pre.TOTAL_SALES.sum()

```

```

scale_customers = t_pre.CUSTOMERS.sum() / c_pre.CUSTOMERS.sum()
scale_txn_per_customer = t_pre.TXNS_PER_CUSTOMER.mean() / c_pre.TXNS_PER_CUSTOMER.mean()
charts.append(plot_comparison(monthly, trial, control, scale_sales, scale_customers))

pair = monthly[monthly.STORE_NBR.isin([trial, control])].pivot(index="MONTH", columns="STORE_NBR",
values=["TOTAL_SALES", "CUSTOMERS", "TXNS_PER_CUSTOMER"])
detail = pd.DataFrame(index=pair.index)
detail["trial_sales"] = pair["TOTAL_SALES"][trial]
detail["control_sales"] = pair["TOTAL_SALES"][control] * scale_sales
detail["sales_uplift"] = detail["trial_sales"] / detail["control_sales"] - 1
detail["trial_customers"] = pair["CUSTOMERS"][trial]
detail["control_customers"] = pair["CUSTOMERS"][control] * scale_customers
detail["customer_uplift"] = detail["trial_customers"] / detail["control_customers"] - 1
detail["trial_txns_per_customer"] = pair["TXNS_PER_CUSTOMER"][trial]
detail["control_txns_per_customer"] = pair["TXNS_PER_CUSTOMER"][control] * scale_txn_per_customer
detail["txn_per_customer_uplift"] = detail["trial_txns_per_customer"] / detail["control_txns_per_customer"] -
1

pre_detail = detail.loc[PRE_START:PRE_END]
trial_detail = detail.loc[TRIAL_START:TRIAL_END]
# Prediction-style threshold based on normal pre-trial variation around the scaled control.
sales_sd = pre_detail["sales_uplift"].abs().std(ddof=1)
customer_sd = pre_detail["customer_uplift"].abs().std(ddof=1)
# One-sided 95% threshold because the test hypothesis is positive uplift.
critical = student_t.ppf(.95, df=len(pre_detail)-1)
avg_sales_uplift = trial_detail["trial_sales"].sum() / trial_detail["control_sales"].sum() - 1
avg_customer_uplift = trial_detail["trial_customers"].sum() / trial_detail["control_customers"].sum() - 1
avg_txn_uplift = trial_detail["trial_txns_per_customer"].mean() /
trial_detail["control_txns_per_customer"].mean() - 1
# Require statistical evidence and a 10% commercial-materiality floor;
# this avoids classifying marginal, method-sensitive differences as wins.
sales_month_threshold = max(critical * sales_sd, 0.10)
sales_sig_months = int((trial_detail["sales_uplift"] > sales_month_threshold).sum())
customer_sig_months = int((trial_detail["customer_uplift"] > critical * customer_sd).sum())
# Match the case-study decision rule: sales must exceed the confidence
# threshold in at least two of the three trial months.
sales_success = sales_sig_months >= 2
customer_success = customer_sig_months >= 2
recommendation = "ROLL OUT" if sales_success else "DO NOT ROLL OUT / INVESTIGATE"
result_rows.append({
    "TRIAL_STORE": trial, "CONTROL_STORE": control, "MATCH_SCORE": row.SCORE,
    "SALES_UPLIFT": avg_sales_uplift, "CUSTOMER_UPLIFT": avg_customer_uplift,
    "TXN_PER_CUSTOMER_UPLIFT": avg_txn_uplift,
    "SALES_THRESHOLD": critical * sales_sd, "CUSTOMER_THRESHOLD": critical * customer_sd,
    "SALES_SIGNIFICANT": sales_success, "CUSTOMER_SIGNIFICANT": customer_success,
    "SALES_SIGNIFICANT_MONTHS": sales_sig_months,
    "CUSTOMER_SIGNIFICANT_MONTHS": customer_sig_months,
    "RECOMMENDATION": recommendation,
})
detail = trial_detail.reset_index()
detail["TRIAL_STORE"], detail["CONTROL_STORE"] = trial, control
monthly_details.append(detail)

results = pd.DataFrame(result_rows)
results.to_csv(OUTPUT_RESULTS, index=False)
details = pd.concat(monthly_details, ignore_index=True)

# PDF.
styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name="TitleX", parent=styles["Title"], fontSize=22, leading=26,
    textColor=colors.HexColor("#17365D"), alignment=TA_CENTER))
styles.add(ParagraphStyle(name="H1X", parent=styles["Heading1"], fontSize=16, leading=19,
    textColor=colors.HexColor("#17365D"), spaceAfter=8))
styles.add(ParagraphStyle(name="H2X", parent=styles["Heading2"], fontSize=12, leading=15,
    textColor=colors.HexColor("#2F6B8A"), spaceAfter=6))
styles.add(ParagraphStyle(name="BodyX", parent=styles["BodyText"], fontSize=9, leading=13, spaceAfter=7))
doc = SimpleDocTemplate(str(OUTPUT_PDF), pagesize=landscape(A4), rightMargin=14*mm,
    leftMargin=14*mm, topMargin=13*mm, bottomMargin=13*mm)
story = [Spacer(1, 30*mm), Paragraph("Quantum Retail Analytics", styles["TitleX"]),
    Paragraph("Task 2 – Experimentation and Uplift Testing", styles["TitleX"]),
    Spacer(1, 8*mm), Paragraph("Matched-control evaluation of trial stores 77, 86, and 88", styles["BodyX"]),
PageBreak()]

display = results[["TRIAL_STORE", "CONTROL_STORE", "MATCH_SCORE", "SALES_UPLIFT", "CUSTOMER_UPLIFT",
    "TXN_PER_CUSTOMER_UPLIFT", "RECOMMENDATION"]].copy()
display["MATCH_SCORE"] = display["MATCH_SCORE"].map(lambda x: f"{x:.3f}")
display["SALES_UPLIFT"] = display["SALES_UPLIFT"].map(lambda x: f"{x:+.1%}")
display["CUSTOMER_UPLIFT"] = display["CUSTOMER_UPLIFT"].map(lambda x: f"{x:+.1%}")
display["TXN_PER_CUSTOMER_UPLIFT"] = display["TXN_PER_CUSTOMER_UPLIFT"].map(lambda x: f"{x:+.1%}")
story += [Paragraph("Executive recommendation", styles["H1X"]),

```

```

        Paragraph("Roll out only where the evidence is repeatable and commercially meaningful. Store-level
results differ, so a blanket national rollout is not supported by this three-store trial.", styles["BodyX"]),
        table(display, widths=[22*mm, 25*mm, 27*mm, 27*mm, 30*mm, 39*mm, 47*mm], font=7.5, Spacer(1, 5*mm))
for row in results.itertuples():
    if row.RECOMMENDATION == "ROLL OUT":
        action = "The sales uplift exceeds normal pre-trial variation; proceed to a controlled expansion while
monitoring margin and repeat behaviour."
    else:
        action = "The uplift was not sufficiently consistent across sales and customers; do not roll it out in its
current form, and retest if operational evidence warrants it."
    story.append(Paragraph(f"<b>Store {row.TRIAL_STORE}</b> {action}", styles["BodyX"]))
story += [Paragraph("Decision rule", styles["H2X"]),
        Paragraph("A trial is successful when positive sales uplift clears both the one-sided 95% threshold and
a 10% commercial-materiality floor in at least two of the three trial months. The materiality floor prevents
small, method-sensitive differences from driving rollout decisions. Customer count and transactions per customer
diagnose the change.", styles["BodyX"]), PageBreak()]

story += [Paragraph("Control-store selection", styles["H1X"]),
        Paragraph("Candidate controls required complete observations for all seven pre-trial months (July
2018-January 2019). Each candidate was scored equally on sales correlation, customer correlation, sales magnitude
similarity, and customer magnitude similarity. Trial stores were excluded from the control pool.",
styles["BodyX"])]
top_candidates = scores.groupby("TRIAL_STORE").head(5)[["TRIAL_STORE", "CONTROL_STORE", "SALES_CORR",
"CUSTOMER_CORR", "SCORE"]].copy()
for c in ["SALES_CORR", "CUSTOMER_CORR", "SCORE"]:
    top_candidates[c] = top_candidates[c].map(lambda x: f"{x:.3f}")
story += [table(top_candidates, widths=[28*mm, 32*mm, 35*mm, 42*mm, 28*mm], font=8),
        Spacer(1, 4*mm), Paragraph("Using both trend and scale prevents selecting a store that moves similarly
but operates at a materially different level. The top-ranked store for each trial is used below.",
styles["BodyX"]), PageBreak()]

for chart, row in zip(charts, results.itertuples()):
    trial_months = details[details.TRIAL_STORE.eq(row.TRIAL_STORE)].copy()
    trial_months["MONTH"] = trial_months["MONTH"].dt.strftime("%Y-%m")
    shown = trial_months[["MONTH", "trial_sales", "control_sales", "sales_uplift", "trial_customers",
"control_customers", "customer_uplift"]].copy()
    for c in ["trial_sales", "control_sales"]:
        shown[c] = shown[c].map(lambda x: f"${x:,.0f}")
    for c in ["trial_customers", "control_customers"]:
        shown[c] = shown[c].map(lambda x: f"{x:,.0f}")
    for c in ["sales_uplift", "customer_uplift"]:
        shown[c] = shown[c].map(lambda x: f"{x:+.1%}")
    story += [Paragraph(f"Trial store {row.TRIAL_STORE} vs control {row.CONTROL_STORE}", styles["H1X"]),
            Image(str(chart), width=190*mm, height=125*mm), Spacer(1, 3*mm),
            table(shown, widths=[24*mm, 30*mm, 34*mm, 30*mm, 38*mm, 35*mm], font=7),
            Paragraph(f"Trial-period sales uplift: <b>{row.SALES_UPLIFT:+.1%}</b>; customer uplift:
<b>{row.CUSTOMER_UPLIFT:+.1%}</b>. Recommendation: <b>{row.RECOMMENDATION}</b>.", styles["BodyX"]), PageBreak()]

story += [Paragraph("Limitations and next steps", styles["H1X"]),
        Paragraph("This observational matched-control design reduces—but cannot eliminate—confounding. The
dataset lacks margin, promotion, stock availability, store format, local events, and competitor activity. Only
three trial months and three trial stores are available, so statistical power and generalisability are limited.",
styles["BodyX"]),
        Paragraph("Before wider rollout, verify gross-margin uplift, check that results persist after promotions
end, review operational execution, and expand with randomised or staggered test/control assignment. Track sales,
customers, transactions per customer, units per transaction, and price per unit.", styles["BodyX"]), PageBreak(),
        Paragraph("Appendix: complete Python code", styles["H1X"])]
source = Path(__file__).read_text(encoding="utf-8")
wrapped = []
for line in source.splitlines():
    wrapped.extend([line] if len(line) <= 118 else textwrap.wrap(line, 118, subsequent_indent="    ",
break_long_words=False, break_on_hyphens=False))
story.append(Preformatted("\n".join(wrapped), ParagraphStyle(name="CodeX", fontName="Courier", fontSize=5.1,
leading=6.3, textColor=colors.HexColor("#1F2933"))))
doc.build(story, onFirstPage=page_number, onLaterPages=page_number)

print("Selected controls:")
print(best[["TRIAL_STORE", "CONTROL_STORE", "SCORE"]].to_string(index=False))
print("\nTrial results:")
print(results.to_string(index=False))
print(f"\nWrote {OUTPUT_PDF}")
print(f"Wrote {OUTPUT_RESULTS}")

if __name__ == "__main__":
    main()

```